

Software Requirements Specification

SIGCB-QR

Sistema Integral de Gestion Bibliotecaria

Version: v0.9.0-rc | Fecha: 2026-07-30

Estado: Release Candidate

Conforme a: ISO/IEC/IEEE 29148:2018

Proyecto de Fin de Ciclo - Aplicaciones Web

Universidad Tecnica Estatal de Quevedo (UTEQ)

PPA 2026-2027

SIGCB-QR -- Sistema Integral de Gestion Bibliotecaria

Version: v0.9.0-rc

Fecha: 2026-07-30

Conforme a: ISO/IEC/IEEE 29148:2018

1. Introduccion

1.1 Proposito

Este documento especifica los requisitos funcionales y no funcionales del Sistema Integral de Gestion, Control y Seguimiento de Lectura Interna en la Biblioteca Universitaria mediante Codigos QR (SIGCB-QR). El sistema automatiza los procesos bibliotecarios: autenticacion, gestion de catalogo, prestamos, reservas, multas y reportes.

1.2 Alcance

SIGCB-QR es una aplicacion web de arquitectura cliente-servidor con backend Spring Boot 3.4, frontend Laravel 13, base de datos PostgreSQL 16 y cache Redis 7. El sistema expone una API REST documentada con OpenAPI 3.0 y utiliza autenticacion stateless JWT en cookie HttpOnly.

1.3 Definiciones

Termino	Definicion
JWT	JSON Web Token (RFC 7519)
JPA	Jakarta Persistence API
ORM	Object-Relational Mapping
CRUD	Create, Read, Update, Delete
TTL	Time To Live
Hit ratio	Proporcion de aciertos de cache

1.4 Referencias

ISO/IEC/IEEE 29148:2018 -- Ingenieria de requisitos
RFC 7519 -- JSON Web Token
RFC 7807 -- Problem Details for HTTP APIs
INCOSE Guide to Writing Requirements v4
OWASP Cheat Sheet -- SQL Injection Prevention
Cockburn -- Writing Effective Use Cases

1.5 Resumen

El SRS se organiza en: descripcion global (seccion 2), requisitos especificos funcionales (3), no funcionales (4), de interfaz externa (5), de seguridad (6) y de calidad (7).

2. Descripcion global

2.1 Perspectiva del producto

SIGCB-QR es un sistema independiente que reemplaza procesos manuales bibliotecarios. Se compone de:

API REST (sigcb-qr-api): Java 21, Spring Boot 3.4.4
Frontend (SIGCB-QR): Laravel 13, PHP 8.4
Base de datos: PostgreSQL 16

Cache: Redis 7

Contenedores: Docker Compose

2.2 Funciones del sistema

Autenticacion stateless con JWT (login, registro, logout)

CRUD de usuarios (admin)

CRUD de material bibliografico (libros, autores, editoriales, categorias)

Gestion de ejemplares (inventario)

Prestamos, devoluciones y renovaciones

Reservas

Multas y sanciones

Reportes y estadisticas de dashboard

Catalogacion (facultades, carreras)

2.3 Caracteristicas de usuarios

Rol	Descripcion
ADMIN	Administracion completa del sistema
BIBLIOTECARIO	Gestion de prestamos, catalogo y reportes
ESTUDIANTE	Consulta de catalogo, prestamos y reservas

2.4 Restricciones

Autenticacion stateless (sin sesiones HTTP)

JWT transmitido en cookie HttpOnly + Secure + SameSite=Strict

Base de datos gestionada exclusivamente por Flyway (ddl-auto=validate)

Operaciones no elementales mediante stored procedures

Contenedores Docker anclados por digest SHA256

2.5 Supuestos y dependencias

Disponibilidad de Docker y Docker Compose

Conexion de red entre contenedores

Semillas deterministas fijadas

3. Requisitos especificos -- Funcionales

REQ-F-001 -- Autenticacion de usuarios

Atributo	Valor
Tipo	Funcional
Prioridad (MoSCoW)	Must
Enunciado	Al recibir credenciales validas, el sistema debera emitir un JWT firmado en cookie HttpOnly
Rationale	La autenticacion stateless elimina la necesidad de almacenar sesiones en servidor
Origen	Stakeholder: Administracion bibliotecaria
Criterio de aceptacion	Login exitoso en menos de 2s con JWT valido
Metodo de verificacion	Test (AuthControllerTest)
Trazabilidad	HU-01, CU-01

REQ-F-002 -- Registro de usuarios

Atributo	Valor
Tipo	Funcional

Prioridad	Must
Enunciado	Al recibir datos de registro validos, el sistema debera crear un usuario con rol ESTUDIAN
Rationale	Permitir auto-registro de estudiantes
Criterio de aceptacion	Registro exitoso con validacion de email unico
Metodo de verificacion	Test
Trazabilidad	HU-02, CU-01

REQ-F-003 -- Cierre de sesion

Atributo	Valor
Tipo	Funcional
Prioridad	Must
Enunciado	Al recibir una solicitud de logout, el sistema debera agregar el JTI del token a la blacklist
Rationale	Garantizar que el token no pueda reutilizarse
Criterio de aceptacion	Token blacklistado no es valido para peticiones posteriores
Metodo de verificacion	Test
Trazabilidad	HU-03, CU-01

REQ-F-004 -- CRUD de usuarios (Admin)

Atributo	Valor
Tipo	Funcional
Prioridad	Must
Enunciado	El sistema debera permitir al administrador crear, leer, actualizar y desactivar usuarios
Rationale	Gestion centralizada de cuentas
Criterio de aceptacion	CRUD completo con validaciones
Metodo de verificacion	Test
Trazabilidad	HU-04

REQ-F-005 -- CRUD de material bibliografico

Atributo	Valor
Tipo	Funcional
Prioridad	Must
Enunciado	El sistema debera permitir crear, leer, actualizar y desactivar libros con sus metadatos (ti
Rationale	Catalogacion del acervo bibliotecario
Criterio de aceptacion	CRUD completo con paginacion y busqueda
Metodo de verificacion	Test
Trazabilidad	HU-05, CU-02

REQ-F-006 -- Prestamo de ejemplares

Atributo	Valor
Tipo	Funcional
Prioridad	Must
Enunciado	Al recibir una solicitud de prestamo valida, el sistema debera crear el prestamo, actualiza
Rationale	Control de circulacion de material
Criterio de aceptacion	Prestamo creado en menos de 2s, maximo 5 activos por usuario
Metodo de verificacion	Test + stored procedure
Trazabilidad	HU-06, CU-03

REQ-F-007 -- Devolucion de prestamo

Atributo	Valor
Tipo	Funcional
Prioridad	Must
Enunciado	Al recibir una devolucion, el sistema debera actualizar el estado del prestamo, liberar el e
Rationale	Control de inventario y penalizaciones
Criterio de aceptacion	Devolucion procesada en menos de 2s
Metodo de verificacion	Test + stored procedure
Trazabilidad	HU-07, CU-04

REQ-F-008 -- Reporte de prestamos diarios

Atributo	Valor
Tipo	Funcional
Prioridad	Should
Enunciado	El sistema debera generar un reporte de prestamos del dia actual con datos de usuario y
Rationale	Seguimiento de actividad diaria
Criterio de aceptacion	Reporte generado en menos de 3s
Metodo de verificacion	Test + stored procedure
Trazabilidad	HU-08

4. Requisitos no funcionales

REQ-NF-001 -- Rendimiento -- Listado de libros

Atributo	Valor
Tipo	No funcional
Prioridad	Must
Enunciado	El endpoint GET /api/libros debera responder con p95 <= 200 ms con cache Redis caliente
Rationale	Experiencia de usuario fluida en navegacion de catalogo
Criterio de aceptacion	p95 < 200ms en 3 corridas con cache caliente
Metodo de verificacion	Test de rendimiento (k6)
Trazabilidad	Evidencia: docs/mediciones/perf/

REQ-NF-002 -- Seguridad -- Autenticacion

Atributo	Valor
Tipo	No funcional
Prioridad	Must
Enunciado	El sistema debera implementar autenticacion JWT sin sesiones, con cookie HttpOnly + S
Rationale	Mitigacion de ataques XSS y CSRF
Criterio de aceptacion	Ningun endpoint protegido accesible sin JWT valido
Metodo de verificacion	Test de seguridad
Trazabilidad	OWASP Top 10

REQ-NF-003 -- Seguridad -- Proteccion contra inyeccion SQL

Atributo	Valor
Tipo	No funcional
Prioridad	Must
Enunciado	Ninguna consulta SQL debera concatenar entrada de usuario; toda parametrizacion sera
Rationale	Prevencion de inyeccion SQL

Criterio de aceptacion	Escaneo SAST sin hallazgos de inyeccion SQL
Metodo de verificacion	Analisis estatico (inspection)
Trazabilidad	OWASP Cheat Sheet

REQ-NF-004 -- Disponibilidad -- Contenedores

Atributo	Valor
Tipo	No funcional
Prioridad	Must
Enunciado	El sistema debera poder reconstruirse en menos de 2 minutos desde una clonacion limpia
Rationale	Reproducibilidad por terceros
Criterio de aceptacion	docker-compose up exitoso en ambiente limpio
Metodo de verificacion	Demostracion (demonstration)
Trazabilidad	Docker Compose

REQ-NF-005 -- Mantenibilidad -- Cobertura de pruebas

Atributo	Valor
Tipo	No funcional
Prioridad	Should
Enunciado	El backend debera tener cobertura de linea >= 30% medida por JaCoCo
Rationale	Garantia minima de calidad
Criterio de aceptacion	Reporte JaCoCo >= 30%
Metodo de verificacion	Test (JaCoCo)
Trazabilidad	pom.xml

5. Requisitos de interfaz externa

5.1 Interfaz de usuario

Frontend Laravel 13 con Tailwind CSS, accesible via navegador en puerto 8000.

5.2 Interfaz API REST

Documentacion OpenAPI 3.0 en /api-docs e interfaz Swagger UI en /swagger-ui.html.

5.3 Interfaz de base de datos

Conexion JDBC a PostgreSQL 16, esquema gestionado por Flyway.

6. Requisitos de seguridad

- JWT con claims estandar RFC 7519
- Cookie HttpOnly + Secure + SameSite=Strict
- Blacklist de JTI revocados en Redis
- CORS restringido a origenes configurados
- BCrypt para almacenamiento de contraseñas
- Parametros nombrados en todas las consultas
- Procedimientos almacenados sin SQL dinamico

7. Calidad de software

7.1 Características del conjunto de requisitos (INCOSE C10-C15)

Atributo	Cumplimiento
----------	--------------

Complete (C10)	Todos los escenarios principales cubiertos
Consistent (C11)	Sin contradicciones entre requisitos
Feasible (C12)	Implementable con la pila tecnologica actual
Comprehensible (C13)	Redactado en lenguaje claro
Able to be validated (C14)	Cada requisito tiene metodo de verificacion
Correct (C15)	Aprobado por stakeholders

Fin del SRS